COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE031	**SECTION:**	CPE21S2
DATE PERFORMED:	05-08-2026	**DATE SUBMITTED:**	05-08-2026
NAME:	GUARIN, JAN MIKAELA C.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 5.1 Queues

1. PROCEDURE OUTPUT

ILO A: Create queue using C++ STL

Screenshot

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);
int main(){
    //create an object
    std::queue<char> myQ;

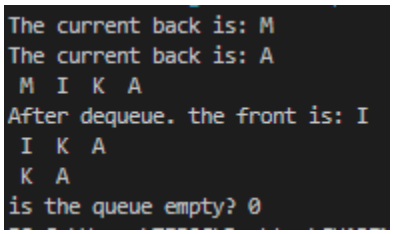
    //use the enqueue operation
    myQ.push('M');
    myQ.push('I');
    myQ.push('K');
    myQ.push('A');

    std::cout<<"The current back is: "<<myQ.front()<<std::endl;
    std::cout<<"The current back is: "<<myQ.back()<<std::endl;
    display(myQ);

    //use the dequeue operation
    myQ.pop();
    std::cout<<"After dequeue. the front is: "<<myQ.front()<<std::endl;
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();

    //check if queue empty
    std::cout<<"is the queue empty? "<<myQ.empty()<<std::endl;
    return 0;
} <- #5-30 int main()

//note only use the member functions of the queue STL
void display(std::queue<char> copyQ){
    //create a copy of the queue
    std::queue<char> temp = copyQ;
    //loop until empty
    while (!temp.empty()){
        //display the front
        std::cout<<" "<<temp.front()<<" ";
        //display the back
        temp.pop();
    } <- #37-42 while (!temp.empty())
    //add a new line
    std::cout <<std::endl;
} <- #33-45 void display(std::queue<char> copyQ)
```

Output	
OBservation	The program implements a queue using a linked list and follows the FIFO (First In, First Out) principle. The enqueue()function inserts new elements at the front, while dequeue() removes elements from the front. The displayAll() function traverses the queue to print all stored elements, and isEmpty() checks if the queue contains any nodes.

ILO B.1. Linked List Implementation

Screenshot	

```
#include<iostream>
#include"queuell.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    std::cout<<"====TESTING THE ENQUEUE OPERATOR=====\n";
    enqueue ('M', &front, &back);
    std::cout<<"FRONT: "<< front->data <<" "<<"BACK: "<<back->data<<std::endl;
    enqueue('I',  &front, &back);
    std::cout<<"FRONT: "<< front->data <<" "<<"BACK: "<<back->data<<std::endl;
    enqueue ('K', &front, &back);
    std::cout<<"FRONT: "<< front->data <<" "<<"BACK: "<<back->data<<std::endl;
    enqueue('A',  &front, &back);
    std::cout<<"FRONT: "<< front->data <<" "<<"BACK: "<<back->data<<std::endl;

    std::cout << "====QUEUE CONTENTS==== \n";
    displayAll(front);

    std::cout << "====CHECKING IF QUEUE IS EMPTY/NOT EMPTY==== \n";
    if(isEmpty(front)){
    std::cout << "Queue is empty.\n";
    }
    else{
        std::cout << "Queue is not empty.\n";
    }

    std::cout<<"====TESTING THE DEQUEUE OPERATOR=====\n";
    dequeue(&front, &back);
    std::cout<<"FRONT: "<< front->data <<" "<<"BACK: "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<"FRONT: "<< front->data <<" "<<"BACK: "<<back->data<<std::endl;
    dequeue(&front, &back);
    dequeue(&front, &back);

    std::cout << "====CHECKING IF QUEUE IS EMPTY/NOT EMPTY==== \n";
    if(isEmpty(front)){
    std::cout << "Queue is empty.\n";
    }
    else{
    std::cout << "Queue is not empty.\n";
    }

    return 0;
} <- #4-46 int main()
```

Figure 1. *main.cpp*

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

//create a class node
template <typename T>
class qNode{
    public:
        T data;
        qNode* next;
}; <- #6-10 class qNode
//new node creator
template <typename T>
qNode<T>* new_node(T newData){
    //allocate a space for new node
    qNode<T>* newNode = new qNode<T>;

    //store the new data to the new node
    newNode -> data = newData;

    //point the new node to null
    newNode -> next = nullptr;

    return newNode;

} <- #13-25 qNode<T>* new_node(T newData)
//enqueue

template <typename T>
void enqueue (T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //create a new node
    qNode<T>* newNode = new_node(newData);

    //check if the queue if empty
    if ((*frontPtr)==nullptr && (*backPtr)== nullptr){
        (*frontPtr)= newNode;
        (*backPtr)= newNode;
        return;

    } <- #34-39 if ((*frontPtr)==nullptr && (*backPtr)== nullptr)
    //if not then insert at the back
    (*backPtr)->next = newNode;
    (*backPtr)=newNode;

} <- #29-44 void enqueue (T newData, qNode<T>** frontPtr, qNode<T>** ba

//dequeue
template <typename T>
void dequeue (qNode<T>** frontPtr, qNode<T>** backPtr){
    //check if the queue if empty
    if ((frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout<<"The queue is empty"<<std::endl;
        return;
    }
    //create a temporary variable to store the node to be deleted
    qNode<T>* deleteTemp = nullptr;

    //assign the current front to deleteTemp
    deleteTemp = (*frontPtr);
```

```
    //create a temporary variable to store the node to be deleted
    qNode<T>* deleteTemp = nullptr;

    //assign the current front to deleteTemp
    deleteTemp = (*frontPtr);

    //check if the queue has only 1 item
    if ((*frontPtr)->next == nullptr && (*backPtr)-> next == nullptr){
        //means only 1 item is inside the queue
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteTemp;
        return;
    } <- #61-67 if ((*frontPtr)->next == nullptr && (*backPtr)-> next == null...
    (*frontPtr) = (*frontPtr)->next;
    deleteTemp->next = nullptr;

    //delete a node
    delete deleteTemp;

} <- #48-74 void dequeue (qNode<T>** frontPtr, qNode<T>** backPtr)

//displayall
template <typename T>
void displayAll(qNode<T> *frontPtr){
    //check if the queue is empty
    if (frontPtr == nullptr){
        std::cout<<"The queue is empty."<<std::endl;
        return;
    }

    //create a temporary pointer
    qNode<T>* temp = frontPtr;

    //display all elements
    while (temp!=nullptr){
        std::cout<<temp-> data<< " ";
        temp = temp-> next;
    }

    std::cout<<std::endl;
} <- #78-95 void displayAll(qNode<T> *frontPtr)

//isEmpty
template <typename T>
bool isEmpty(qNode<T>* frontPtr){
    //check if the queue is empty
    if (frontPtr == nullptr){
        return true;
    }

    return false;
} <- #99-106 bool isEmpty(qNode<T>* frontPtr)
```

Figure 2. *queuell.h*

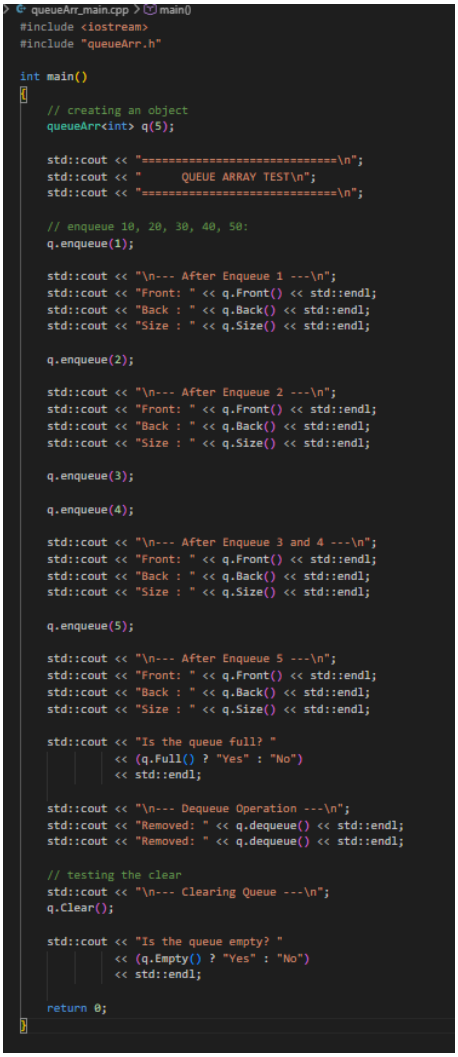
Output

```
====TESTING THE ENQUEUE OPERATOR=====
FRONT: M BACK: M
FRONT: M BACK: I
FRONT: M BACK: K
FRONT: M BACK: A
====QUEUE CONTENTS====
M I K A
====CHECKING IF QUEUE IS EMPTY/NOT EMPTY====
Queue is not empty.
====TESTING THE DEQUEUE OPERATOR=====
FRONT: I BACK: A
FRONT: K BACK: A
====CHECKING IF QUEUE IS EMPTY/NOT EMPTY====
Queue is empty.
```

OBservation

From what I observed, the STL queue made it easier to perform queue operations because the built-in functions handled most of the work. The push() function added elements to the rear, while pop() removed the front element following the FIFO (First In, First Out) principle. I also noticed that the display() function used a copy of the queue so the original queue was not modified while displaying its contents. The empty() function also helped in checking whether there were still elements left in the queue after performing the dequeue operations.

ILO B.2 B.2. Array-Based Implementation

Screenshot	

queueArr_main.cpp > main()

```cpp
#include <iostream>
#include "queueArr.h"

int main()
{
    // creating an object
    queueArr<int> q(5);

    std::cout << "============================\n";
    std::cout << "      QUEUE ARRAY TEST\n";
    std::cout << "============================\n";

    // enqueue 10, 20, 30, 40, 50:
    q.enqueue(1);

    std::cout << "\n--- After Enqueue 1 ---\n";
    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back : " << q.Back() << std::endl;
    std::cout << "Size : " << q.Size() << std::endl;

    q.enqueue(2);

    std::cout << "\n--- After Enqueue 2 ---\n";
    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back : " << q.Back() << std::endl;
    std::cout << "Size : " << q.Size() << std::endl;

    q.enqueue(3);

    q.enqueue(4);

    std::cout << "\n--- After Enqueue 3 and 4 ---\n";
    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back : " << q.Back() << std::endl;
    std::cout << "Size : " << q.Size() << std::endl;

    q.enqueue(5);

    std::cout << "\n--- After Enqueue 5 ---\n";
    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back : " << q.Back() << std::endl;
    std::cout << "Size : " << q.Size() << std::endl;

    std::cout << "Is the queue full? "
              << (q.Full() ? "Yes" : "No")
              << std::endl;

    std::cout << "\n--- Dequeue Operation ---\n";
    std::cout << "Removed: " << q.dequeue() << std::endl;
    std::cout << "Removed: " << q.dequeue() << std::endl;

    // testing the clear
    std::cout << "\n--- Clearing Queue ---\n";
    q.Clear();

    std::cout << "Is the queue empty? "
              << (q.Empty() ? "Yes" : "No")
              << std::endl;

    return 0;
}
```

Figure 1. queueArr_main.cpp

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

// Array-Based Circular Queue
template <typename T>
class queueArr
{
private:
    // POINTER TO DYNAMICALLY ALLOCATE ARRAY
    T *q_arr;

    // MAXIMUM NUMBER OF ELEMENTS A QUEUE CAN HOLD
    size_t q_capacity;

    // CURRENT NUMBER OF ELEMENTS IN THE QUEUE
    size_t q_size;

    // INDEX OF THE FRONT ELEMENT IN THE QUEUE
    int q_front;

    // INDEX OF THE BACK ELEMENT IN THE QUEUE
    int q_back;

public:
    // CONSTRUCTOR
    queueArr(size_t capacity) : q_capacity(capacity), q_size(0), q_front(0), q_back(-1)
    {
        // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
        q_arr = new T[q_capacity];
    }

    // COPY CONSTRUCTOR
    queueArr(const queueArr &other) : q_capacity(other.q_capacity), q_size(other.q_size), q_front(other.q_front), q_back(other.q_back)
    {
        // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
        q_arr = new T[q_capacity];
        // COPY ELEMENTS FROM OTHER QUEUE
        for (size_t i = 0; i < q_size; ++i)
        {
            q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
        }
    }

    // COPY ASSIGNMENT OPERATOR
    queueArr &operator=(const queueArr &other)
    {
        if (this != &other)
        {
            // DEALLOCATE CURRENT ARRAY
            delete[] q_arr;

            // COPY CAPACITY AND SIZE
            q_capacity = other.q_capacity;
            q_size = other.q_size;
            q_front = other.q_front;
            q_back = other.q_back;

            // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
            q_arr = new T[q_capacity];
            // COPY ELEMENTS FROM OTHER QUEUE
            for (size_t i = 0; i < q_size; ++i)
            {
                q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
            }
        }
        return *this;
    }

    // DESTRUCTOR
```

```
    // DESTRUCTOR
    ~queueArr()
    {
        // DEALLOCATE MEMORY FOR THE QUEUE ARRAY
        delete[] q_arr;
    }

    // QUEUE OPERATIONS
    bool Empty();
    bool Full();
    size_t Size();
    void Clear();
    T Front();
    T Back();
    void enqueue(T value);
    T dequeue();
};

// Empty()
template <typename T>
bool queueArr<T>::Empty()
{
    // CHECK IF EMPTY
    return q_size == 0;
}

// Full()
template <typename T>
bool queueArr<T>::Full()
{
    // CHECK IF REACHES MAX CAPACITY
    return q_size == q_capacity;
}

// Size()
template <typename T>
size_t queueArr<T>::Size()
{
    // RETURN CURRENT SIZE OF QUEUE
    return q_size;
}

// Front()
template <typename T>
T queueArr<T>::Front()
{
    // Check if the queue is empty
    if (Empty())
    {
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    // Return the front
    return q_arr[q_front];
}

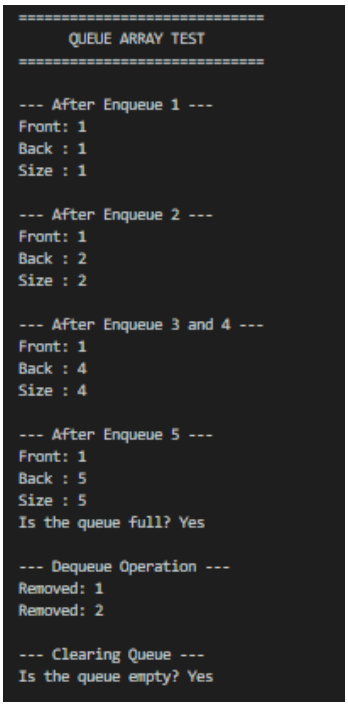
// Back()
template <typename T>
T queueArr<T>::Back()
{
    // Check if the queue is empty
    if (Empty())
    {
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    // Return the back
```

```cpp
// Enqueue()
template <typename T>
void queueArr<T>::enqueue(T value)
{
    // Check if the queue is full
    if (Full())
    {
        std::cout << "Queue is Full\n";
        return;
    }
    // Increment the back
    q_back = (q_back + 1) % q_capacity;
    // Add the new value to the back of the queue
    q_arr[q_back] = value;
    // Increment the size of the queue
    q_size++;
}

// Dequeue()
template <typename T>
T queueArr<T>::dequeue()
{
    // Check if the queue is empty
    if (Empty())
    {
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    // Store the front value
    T value = q_arr[q_front];
    // Increment the front index in a circular manner
    q_front = (q_front + 1) % q_capacity;
    // Decrement the size of the queue
    q_size--;
    // Reset the indexes
    if (Empty())
        Clear();
    // Return the dequeued value
    return value;
}

#endif // QUEUEARR_H
```

Figure 2. queueAr.h

Output	
Observation	The program successfully implemented an array-based circular queue. I observed that the enqueue() function adds elements at the rear of the queue, while the dequeue() function removes elements from the front following the FIFO (First-In, First-Out) principle. The Front(), Back(), and Size() functions help display the current status of the queue after each operation. I also observed that the circular queue uses indexes (q_front and q_back) to manage the positions of elements and reuse available spaces in the array. The Full() and Empty() functions prevent invalid operations, while Clear() resets the queue back to its original state.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C. Solve Problems Using Implementation

job.h

```
#ifndef JOB_H
#define JOB_H

#include <iostream>
#include <string>

using namespace std;

class Job
{
private:
    int jobID;
    string username;
    int pages;

public:
    Job(int id, string user, int page)
    {
        jobID = id;
        username = user;
        pages = page;
    }

    void display()
    {
        cout << "Job ID: " << jobID << endl;
        cout << "Username: " << username << endl;
        cout << "Pages: " << pages << endl;
    }
};

#endif
```

OBSERVATION:

The file contains the Job class that stores the information of every printing request. It includes the job ID, username, and number of pages, which represent the details needed for each print task. The display() function is used to show the information of a specific job when viewing the printer queue.

printer.h

```
#ifndef PRINTER_H
#define PRINTER_H

#include "Job.h"

class Node
{
public:
    Job data;
    Node *next;

    Node(Job job) : data(job)
    {
        next = NULL;
    }
};

class Printer
{

private:
    Node *front;
    Node *rear;

public:
    Printer()
    {
        front = NULL;
        rear = NULL;
    }

    // Add job into queue
    void addJob(Job job)
    {

        Node *newNode = new Node(job);

        if (front == NULL)
        {
            front = rear = newNode;
        }

        else
        {
            rear->next = newNode;
            rear = newNode;
        }

        cout << "\nPrint Job Added Successfully!\n";
    }

    // Display queue
    void viewQueue()
    {

        if (front == NULL)
        {
            cout << "\nPrinter Queue is Empty.\n";
            return;
        }

        Node *current = front;

        int count = 0;

        cout << "\n========== PRINT QUEUE ==========\n";

        while (current != NULL)
        {

            cout << "\nPosition [" << count << "]\n";
            cout << "--------------------------\n";

            current->data.display();

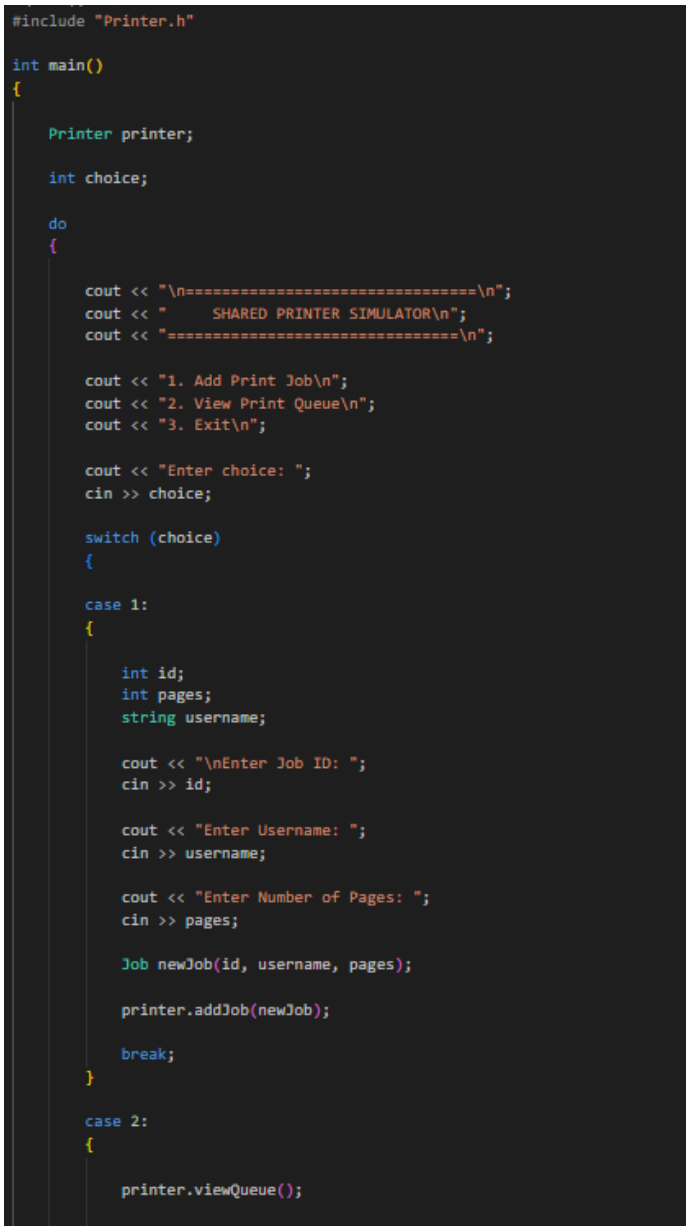
            cout << "--------------------------\n";

            current = current->next;

            count++;
        }

        cout << "================================\n";
    }
};

#endif
```

	OBSERVATION: This file contains the implementation of the queue using a singly linked list. It uses nodes and pointers to connect each printing job in the queue. The addJob() function inserts new print requests at the rear of the queue, while the viewQueue() function displays the jobs starting from the front. I observed how linked lists can be used to create a dynamic queue where elements can be added without a fixed size.
print.cpp	

```cpp
#include "Printer.h"

int main()
{

    Printer printer;

    int choice;

    do
    {

        cout << "\n==============================\n";
        cout << "     SHARED PRINTER SIMULATOR\n";
        cout << "==============================\n";

        cout << "1. Add Print Job\n";
        cout << "2. View Print Queue\n";
        cout << "3. Exit\n";

        cout << "Enter choice: ";
        cin >> choice;

        switch (choice)
        {

        case 1:
        {

            int id;
            int pages;
            string username;

            cout << "\nEnter Job ID: ";
            cin >> id;

            cout << "Enter Username: ";
            cin >> username;

            cout << "Enter Number of Pages: ";
            cin >> pages;

            Job newJob(id, username, pages);

            printer.addJob(newJob);

            break;
        }

        case 2:
        {

            printer.viewQueue();
```

```
            break;
        }

        case 3:
        {

            cout << "\nThank you for using Shared Printer Simulator.\n";

            break;
        }

        default:
        {

            cout << "\nInvalid Choice.\n";
        }
        }

    } while (choice != 3);

    return 0;
}
```

OBSERVATION:
The class serves as the main driver of the program. It provides the menu where the user can add print jobs, view the current queue, and exit the program. It connects the Job and Printer classes to test how the queue works in a real-world printer simulation. I observed that the program properly handles user input and performs the required queue operations.

Output

```
==============================
     SHARED PRINTER SIMULATOR
==============================
1. Add Print Job
2. View Print Queue
3. Exit
Enter choice: 1

Enter Job ID: 45123
Enter Username: kaleia
Enter Number of Pages: 7

Print Job Added Successfully!

==============================
     SHARED PRINTER SIMULATOR
==============================
1. Add Print Job
2. View Print Queue
3. Exit
Enter choice: 1

Enter Job ID: 45753
Enter Username: lhian
Enter Number of Pages: 6

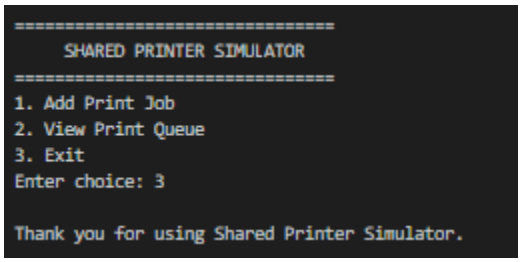
Print Job Added Successfully!
```

```
==============================
     SHARED PRINTER SIMULATOR
==============================
1. Add Print Job
2. View Print Queue
3. Exit
Enter choice: 2

========== PRINT QUEUE ==========

Position [0]
--------------------------
Job ID: 45123
Username: kaleia
Pages: 7
--------------------------

Position [1]
--------------------------
Job ID: 45753
Username: lhian
Pages: 6
--------------------------
==============================
```

	 **OBSERVATION:** The output shows that the shared printer simulation successfully stores and displays multiple printing requests in the order they were added. I observed that the first job entered appears at the front of the queue, while newly added jobs are placed at the rear, demonstrating the FIFO behavior of a queue.

For this **Shared Printer Simulation**, I chose **singly linked list** as the internal representation because it is more suitable for a queue where the number of print jobs is constantly changing. In a real printer system, users can submit different numbers of printing requests at any time, so using a fixed-size array may not be practical because it has a limited capacity.

I chose linked list because it allows new print jobs to be added easily at the rear of the queue and completed jobs to be removed from the front without shifting other elements. This makes the enqueue and dequeue operations simpler while maintaining the FIFO (First-In, First-Out) behavior of the queue.

Although an array can also be used for implementing a queue, it requires managing a fixed amount of memory and additional handling when the queue becomes full. A linked list provides more flexibility because memory is allocated only when a new print job is added. Therefore, it is a better choice for this printer simulation since the number of pending jobs can increase or decrease dynamically.

CONCLUSION

Through this activity, I learned more about the concept and implementation of queues in data structures. I was able to understand how queues follow the FIFO (First-In, First-Out) principle, where the first element added is the first one removed. I learned how enqueue and dequeue operations work and how these operations are applied in both array-based and linked list implementations.

I also learned the importance of choosing the appropriate internal representation depending on the problem. Through the Shared Printer Simulation, I was able to apply the queue concept in a real-world scenario and understand how tasks are managed and processed in order. This activity also helped me improve my understanding of classes, objects, pointers, and how nodes are connected in a linked list.

I think I performed okay in this activity because I was able to understand the basic concepts and complete the required implementations, although I still encountered some challenges while organizing the code and understanding how the operations work internally. I was able to improve my understanding through testing the

program and analyzing the output of each operation.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code Assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used AI for code assistance, such as helping me organize the program structure, checking errors, and improving the readability of my code.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.